From Uncertainty Estimation to Repair Allocation in Code Generation: An Empirical Study

Melih Elibol[a,b,*], Ayşenur Gönül[a,b], Sushant Kumar Pandey[b]

[a] *Sabancı University, Istanbul, Turkey*
[b] *Faculty of Science and Engineering, University of Groningen, Groningen, The Netherlands*

Abstract

Context: Large language models (LLMs) generate code that often parses but fails its tests. Uncertainty estimation and execution-guided repair offer complementary approaches to improving LLM-generated code. A correctness predictor, however, need not identify the failures that benefit most from repair.

Objectives: To test whether one disagreement-based uncertainty signal supports three distinct decisions: (i) predicting whether a generated program is correct, (ii) selecting one program among several sampled candidates, and (iii) allocating a limited repair budget over observed failures.

Methods: We study lexical, structural, embedding-based, and execution-behavior signals on the public HumanEval and MBPP benchmarks, with grouped cross-validation and controls for code length, representation, and runnability. An integrated pipeline combines these signals with instruction-tuned repair. Frozen repair traces support paired policy comparisons, repeated random allocation, and fixed-call prompt ablations. A subset of the public BigCodeBench benchmark tests the scope of the uncertainty findings.

Results: Sample-level text–execution models achieve areas under the receiver operating characteristic curve (ROC-AUC) up to 0.93, while execution-consensus selection reaches 0.700 accuracy against an expected random-selection accuracy of 0.495 on one 60-problem HumanEval cell. Repair corrects 12 of 50 held-out MBPP failures (24.0%). Nevertheless, coverage-matched random policies match or exceed the uncertainty gate's gains at no greater repair-call cost in 82.5% of MBPP and 97.5% of HumanEval replays. At fixed call budgets, regeneration without the failing program achieves 61.5% versus 36.5% success on HumanEval and 50.0% versus 24.1% on MBPP compared with code-plus-error repair.

Conclusion: Correctness estimation, candidate selection, and repair allocation require distinct validation targets. In the evaluated settings, strong correctness ranking does not establish efficient repair allocation, and additional feedback does not consistently justify its cost. These findings support evaluating uncertainty-guided repair against regeneration, random allocation, and inexpensive program features before adopting a more complex policy.

Keywords: code generation, uncertainty estimation, automated program repair, selective computation, empirical software engineering, open-weight language models

*Corresponding author.
Email addresses: melih.elibol@sabanciuniv.edu (Melih Elibol), aysenur.gonul@sabanciuniv.edu (Ayşenur Gönül), s.k.pandey@rug.nl (Sushant Kumar Pandey)

1. Introduction

Large language models (LLMs) generate executable code from a natural-language specification, and that capability is now embedded in everyday development tools [1, 2]. Syntactically valid code is nevertheless not guaranteed to implement the requested behavior: functional correctness has to be established by executing tests rather than by inspecting the program text [3], and strengthening a test suite changes the measured correctness of programs that passed the weaker suite [4]. Two public function-level benchmarks make that gap observable. HumanEval[1] provides 164 hand-written Python programming problems [3], and MBPP (Mostly Basic Python Problems)[2] provides about a thousand crowd-sourced entry-level Python problems [5]. Every problem ships with a specification, a function signature and an executable test suite, so each generated program receives a pass or fail verdict. Throughout this paper a *candidate* is one complete program that an LLM generated for one such problem and that is judged by that problem's tests; because ten programs are sampled per problem, one problem has several candidates.

Two reliability mechanisms act around that verdict. *Uncertainty estimation* acts before a candidate is accepted: it scores how likely the candidate is to be wrong, typically from the disagreement among the candidates sampled for the same problem [6, 7]. *Execution-guided repair* acts after a failure has been observed: it returns the failing candidate, and optionally its error message, to a model that generates a replacement [8, 9]. Composing the two yields a *risk-gated repair* policy: rank the candidates by estimated failure probability and spend a limited repair budget on the riskiest ones first.

Risk-gated repair assumes, without evaluating it, that the ranking which best detects wrong candidates is also the ranking which best allocates repair effort. Two observations show why the two need not coincide. (1) A candidate can be highly likely to fail and still be difficult to repair, so an accurate failure ranking can concentrate the budget on the least tractable failures. (2) Conversely, a candidate that appears comparatively reliable may contain a local defect that a repair model fixes cheaply, and a risk-ordered policy reaches it last or not at all. Detection and allocation are different targets measured over different populations: all candidates in the first case, only the failing ones in the second. The distinction matters because the uncertainty score is not free: it requires multiple generations, execution probes and a learned predictor, and the policy has to repay those costs.

Neither literature closes that gap on its own. Work on sampling-based uncertainty evaluates its signals *as predictors*: SelfCheckGPT scores an output by its consistency with independently sampled alternatives [6]; Lin et al. separate dispersion over outputs from confidence in an individual output [7]; code-specific methods add execution behavior and graded semantic distances between programs [10, 11]; and calibration work asks whether the resulting probabilities are numerically trustworthy [12, 13]. Their endpoints are ranking and calibration quality against correctness labels; where a downstream use is evaluated, it is abstention or reranking among samples already generated [11], so what such a score buys when it allocates further computation, such as repair, is left unmeasured. Work on automated program repair (APR) [14] and its LLM-based successors [8, 9] evaluates the repair step instead, and reports challenges of the opposite kind: gains can become modest or disappear once inference cost is charged against them [15], and test-based acceptance is an imperfect verifier whose errors bound what resampling can deliver [16]. Because repair is customarily compared with no repair, that comparison also absorbs the value of simply generating

[1] https://github.com/openai/human-eval
[2] https://github.com/google-research/google-research/tree/master/mbpp

another solution. Two questions therefore remain open: whether a correctness score that ranks well also allocates well, and whether either component beats the inexpensive controls it must beat before deployment, namely another generation at the same cost, a random allocation of the same budget, and an ordering by a simple property of the failing program.

We address them by building the two components as one pipeline over the same programs (overview in Figure 1, stages in Figure 2). The uncertainty component samples ten programs per problem, measures how far they disagree in surface tokens, abstract syntax structure, a learned embedding and runtime behavior on shared probe inputs (inputs only, never expected outputs, so that no behavioral feature reads the correctness label), and fits a problem-grouped classifier whose out-of-fold score estimates each candidate's probability of passing. The repair component builds a prompt for each failing candidate from the task specification alone, or additionally from the failing program, or additionally from its execution error; generates replacement candidates; executes and ranks them by test outcome; and records every result. Because each failing candidate's repair outcome and call cost are stored once and frozen, any allocation policy can be replayed over identical results, so policy differences are attributable to the allocation decision rather than to generation randomness, and a policy's price is read off in replacement-model calls.

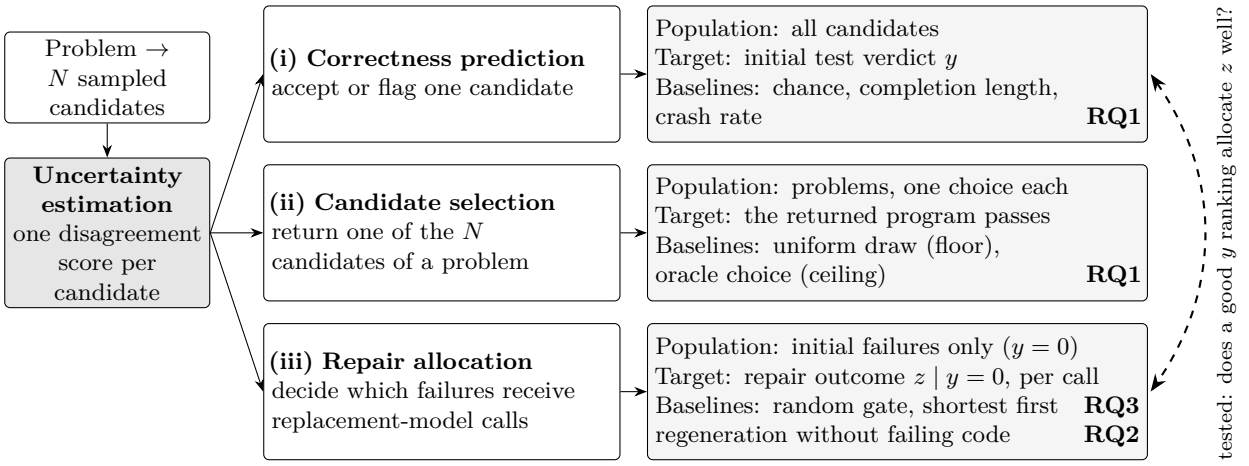


Figure 1: Overview of the study. One disagreement-based uncertainty score is put to three uses, each a different decision with its own population, validation target and inexpensive baseline. The dashed arrow is the assumption that risk-gated repair makes and this study tests: that a score which ranks initial correctness well (i) also allocates repair effort well (iii).

The principal contribution is an empirical account of the relationship between *predicting correctness* and *spending computation to improve correctness*, organized by three research questions that correspond to the three uses in Figure 1.

1. **RQ1: How useful are disagreement-based signals for correctness estimation and candidate selection, after measurement controls?** Disagreement is the cheapest correctness signal available without running tests, but it is computed over strings that share a prompt and differ in length, which an uncontrolled coefficient can credit to the signal. We find that sample-level models combining textual and execution disagreement reach areas under the receiver operating characteristic curve (ROC-AUC) of up to 0.93 and that an execution-consensus rule selects a passing program on 0.700 of one 60-problem HumanEval

cell against an expected random-selection accuracy of 0.495, while length and runnability controls stay competitive at problem level and the most useful representation changes with the benchmark.

2. **RQ2: How much do multi-candidate repair and failure context contribute beyond generating another solution?** Repair versus no repair confounds an extra generation, a different model and failure conditioning; only an arm with the same replacement model and call budget isolates the failure context. We find that repair corrects 12 of 50 held-out MBPP failures (24.0%), yet at equal call budgets regeneration from the task specification alone outperforms code-plus-error repair, 61.5% against 36.5% on HumanEval and 50.0% against 24.1% on MBPP.
3. **RQ3: Does a correctness-based uncertainty ranking allocate repair calls efficiently compared with random and inexpensive alternative policies?** A gate repays its cost only if it beats allocations that need no model, and its training target, initial correctness, is not its use target, repairability. We find that coverage-matched random policies match or exceed the gate's gains at no greater repair-call cost in 82.5% of MBPP and 97.5% of HumanEval replays, and that on HumanEval the score's association with repair success runs opposite to the direction in which the gate consumes it.

The study concerns small open-weight models and bounded Python tasks; it claims neither state-of-the-art performance nor universal inferiority of repair, but identifies when an apparently reasonable reliability policy fails to beat simpler alternatives.

Section 2 reviews related work. Section 3 defines the integrated pipeline and its decision targets. Section 4 maps each research question onto its experiment, cohort, control arm and statistical comparison. Section 5 reports the results per research question, and Section 6 interprets them along the three uses and draws implications for industrial practice. Section 7 states the threats to validity and their mitigation, and Section 8 concludes with directions for follow-up work. Supporting tables are in the supplementary material, Sections S1–S6.

2. Background and related work

Four bodies of work bear on this study, reviewed in the order in which the pipeline consumes them: benchmarks and test oracles, sampling-based uncertainty estimation, selection among sampled candidates, and repair. Table 1 closes the section by separating studies that inform the design from controls matched experimentally inside it.

2.1. Benchmarks and correctness oracles

HumanEval [3] and MBPP [5] evaluate function-level program synthesis with executable tests, but results also depend on the problems, checkpoints, prompts and test versions used. EvalPlus shows that insufficient tests can change estimated performance and model rankings [4], so passing an available suite is treated here as an operational outcome rather than a proof of correctness. BigCodeBench adds tasks requiring more diverse library use and function composition [17]; we use a 60-problem environment-eligible subset as a stress test of the uncertainty findings only, not as a full-benchmark score or an evaluation of the repair system. Repository-level benchmarks such as SWE-bench, in which a model resolves a real GitHub issue inside an existing codebase [18], are outside the scope of this study.

2.2. Uncertainty from repeated generation

Sampling-based methods use variation among outputs as a proxy for uncertainty. SelfCheckGPT compares an output with independently sampled alternatives [6], and Lin et al. distinguish dispersion over outputs from confidence in an individual output [7]. The intuition that answers on which independent samples agree are more often right was first made operational for reasoning by self-consistency decoding [19]; semantic entropy refines it by clustering samples that mean the same thing before measuring dispersion [20] and detects confabulated answers across tasks without task-specific supervision [21]. For code, where programs with little surface overlap can compute the same function, differences can be measured in surface tokens, abstract syntax trees (ASTs), embeddings such as CodeBERT and CodeBERTScore [22, 23], or execution behavior.

Execution-based approaches matter because different implementations can be behaviorally equivalent. He et al. study semantic-distance-aware uncertainty across models, languages and benchmarks [10], and Code-MUE uses execution-based semantic interaction graphs [11]. Functional Entropy addresses equivalence without execution, replacing the natural-language-inference equivalence of semantic entropy with a functional-equivalence judgment by a separate LLM, and reports that inference-based clustering transfers poorly to code [24]; Shi et al. argue that code needs its own uncertainty dimensions, combining token-level, algorithmic and execution-consistency signals [25]. Our execution features belong to this family; we do not claim the invention of behavioral disagreement or graded error distances, but evaluate them alongside representation and runnability controls and in repair allocation. No external method is reimplemented, so the comparison is methodological rather than numerical.

Calibration is a separate issue from ranking: a probability can rank outcomes well while systematically overestimating their likelihood [12]. Beta calibration [13] is used here with nested fitting, and calibration is measured rather than assumed because the confidence of code models is often poorly calibrated against functional correctness, with post-hoc rescaling helping only in some settings [27]. Selective prediction judges a system that abstains on low-confidence inputs by its risk at a given coverage; Traub et al. show that the common area under the risk–coverage curve violates several requirements of a multi-threshold metric and propose a generalized alternative [28].

2.3. Selecting among sampled candidates

When several candidates exist, the decision is which one to return. Minimum-Bayes-risk decoding over execution results selects the candidate whose outputs agree most with the others [29]; CodeT additionally generates test inputs with the model and scores a dual agreement between outputs and tests [30]; DiffCodeGen clusters candidates by their behavior on fuzzed inputs and selects from the largest cluster without reference tests [26] (it is unrelated to He et al. above despite the shared surname); and Code-MUE selects the most central candidate of its execution-similarity graph [11]. Our execution-consensus selector belongs to this family and is evaluated against a uniform draw and the oracle choice, not against their published figures.

2.4. Repair, feedback, and resampling

APR uses evidence about a defect to propose a modification [14, 31] and has been deployed industrially, for example to fix crashes in production systems at scale [32]. Pre-trained language models outperform earlier learning-based repair tools [33], repair the incorrect solutions code models produce [34], and repair conversationally from fed-back test failures at a reported cost of cents per bug [35]. General LLM-based approaches include self-generated feedback, iterative refinement and

debugging with execution information [8, 9]. Olausson et al. show that self-repair gains can be modest or absent once inference cost is considered [15], and resampling with imperfect verifiers cautions against treating test acceptance as infallible [16]. Our main repair setting is *cross-model* (Section 4.1) and is therefore not strictly self-repair; a no-context arm with the same replacement model and budget separates the benefit of another generation from that of conditioning on the failed program.

Table 1: Comparison scope. External studies inform the design; direct numerical comparisons are made only within matched local experiments.

Reference or control	Relevant comparison	Scope in this study
SelfCheckGPT [6]; Lin et al. [7]	Sample disagreement and confidence	Adaptation to code; not a reproduction of their original tasks.
He et al. [10]; Code-MUE [11]	Execution-based uncertainty	Related methods; no matched external implementation.
Olausson et al. [15]	Repair value relative to cost	Motivates controlling model and generation budget.
EvalPlus [4]	Stronger acceptance tests	Stored-program relabeling; not a new generation model.
Fresh generation	Value of failure context	Same replacement model, cohort, and three-call budget.
Random / length allocation	Value of uncertainty ranking	Frozen traces; coverage and call caps distinguished.
Base-model resampling	Value of repair relative to another draw	Stored samples replayed at matched coordinates and matched call count; calls go to the generator, not the replacement model.

3. Integrated pipeline

The *integrated pipeline* is the single system in which the uncertainty component and the repair component operate on the same generated programs (Figure 2). This section defines its decision targets, its features and correctness models, its replacement step, and the accounting that makes allocation policies comparable.

3.1. Information flow and decision targets

For problem j, let $C_j = \{c_{j1}, \ldots, c_{jN}\}$ be N generated programs and let $y_{ji} \in \{0, 1\}$ indicate acceptance by the evaluation suite. An uncertainty model produces a pass score s_{ji} from disagreement features. A repair attempt on a failing candidate has a different outcome, $z_{ji} \in \{0, 1\}$, indicating whether at least one accepted replacement is obtained within its budget.

The expression $z_{ji} \mid y_{ji} = 0$ reads "the repair outcome of candidate i of problem j, considered only for candidates that failed their tests"; it is conditional because a passing candidate is never sent to the replacement model, so for $y_{ji} = 1$ the quantity z_{ji} is undefined rather than zero. The joint MBPP cohort (Section 5.3) is a worked instance. It contains 60 candidates, one per problem; six pass ($y = 1$) and are retained untouched, and the remaining 54 ($y = 0$) are exactly the population on which z is defined. Repairing all 54 yields an accepted program for ten, so $z = 1$ in ten cases and $z = 0$ in 44. Predicting y thus uses 60 rows with six positives, predicting $z \mid y = 0$ 54 different

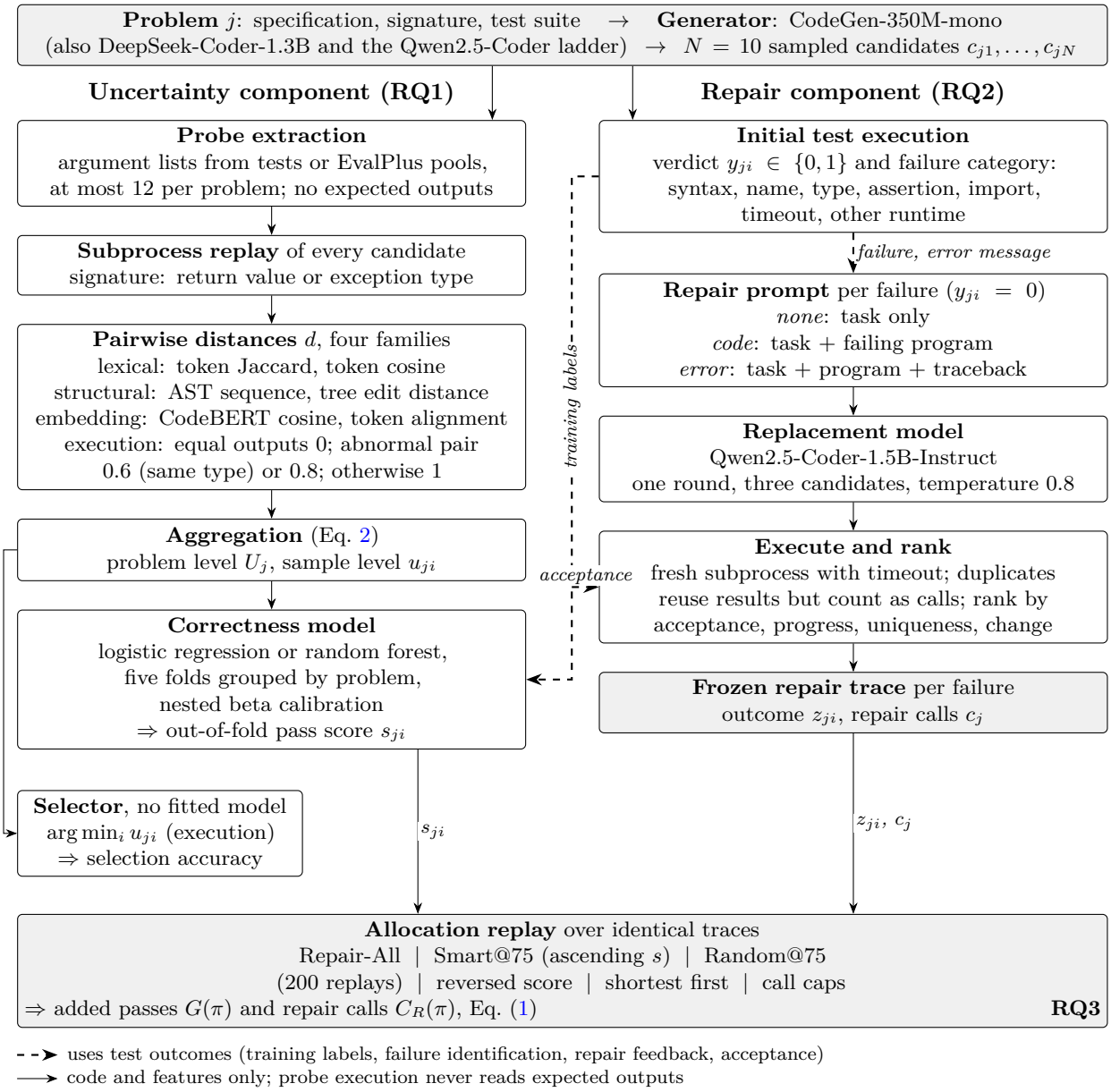


Figure 2: The integrated pipeline. The uncertainty component (left) turns the N sampled programs into a pass score s_{ji} and a selection; the repair component (right) turns an observed failure into a stored replacement outcome z_{ji} and its call cost. The two meet through aligned candidate identifiers in a replay that evaluates every allocation policy over the same frozen traces. Dashed arrows mark every use of test outcomes.

rows with ten different positives: different labels, populations and, as Section 5.3 shows, attainable accuracy.

For an allocation policy π over candidates, the additional accepted outputs and repair-call cost are

$$G(\pi) = \sum_{j \in \pi} (1 - y_{j0}) z_{j0}, \qquad C_R(\pi) = \sum_{j \in \pi} (1 - y_{j0}) c_j, \tag{1}$$

where c_j is the number of repair-model calls in the stored trace. Initially passing candidates are retained and incur no repair calls, so allocation is evaluated in a setting where test outcomes are available; the experiments do not show that uncertainty replaces initial test execution. Test outcomes supply training labels, initial failure identification, repair feedback and candidate acceptance (dashed arrows in Figure 2); probe execution never reads expected outputs.

3.2. Disagreement features and correctness models

For a pairwise distance d, problem-level and sample-level aggregates are

$$U_j = \frac{2}{N(N-1)} \sum_{a<b} d(c_{ja}, c_{jb}), \qquad u_{ji} = \frac{1}{N-1} \sum_{k \neq i} d(c_{ji}, c_{jk}). \tag{2}$$

The lexical family includes token-set Jaccard and token-count cosine distances; a separate unique-sequence statistic measures lexical diversity. Structural metrics compare AST node sequences or use normalized tree edit distance [36]. Embedding metrics use CodeBERT representations [22] and token alignment [23]. Execution features compare return values and exception types on shared probe inputs, which are argument lists extracted from the tests or from EvalPlus input pools, capped at 12 per problem, with expected values and verdicts excluded. Equal normal outcomes have distance zero; different normal outcomes and a normal/abnormal pair have distance one; two abnormal outcomes receive 0.8 for different exception types and 0.6 for the same type, so that repeated crashes do not automatically imply agreement, following the semantic-distance design of prior work [10].

Problem-level prediction asks whether any of the N samples passes; sample-level prediction asks whether one specific sample passes. A problem-level feature such as U_j takes one value per problem, so replicating it across the N samples yields rows that are not independent, and counting them as such overstates the evidence behind a sample-level coefficient. For the same reason a row-wise random split would place near-duplicate rows on both sides of a partition; this is the clustered-data situation for which grouped cross-validation is recommended [37], and a recurring leakage pattern behind over-reported accuracy in machine-learning-based science [38]. All predictors therefore use five-fold splits grouped by problem. Two model families are fitted: logistic regression, which is monotone in each feature and so interpretable as a ranking rule, and random forest, which can express threshold effects. Calibration is fitted inside each outer training fold on that fold's own inner out-of-fold predictions, because fitting a calibrator on predictions the model has already seen makes it appear better calibrated than it is [39]; beta calibration is the parametric form [13].

A candidate selector chooses one of the N programs already sampled for a problem. The execution-consensus heuristic proceeds in three steps. First, every program is replayed on the shared probe inputs and its behavior signature recorded; this is the execution pass the uncertainty features already require. Second, each sample receives its leave-one-out mean behavioral distance u_{ji} (Eq. (2)) under the graded execution distance, so it scores low when its outputs agree with the other $N - 1$ samples on most probes. Third, the selector returns $\arg\min_i u_{ji}$, breaking ties by

sample index. No correctness label, expected output or test verdict enters any step. The rule needs no fitted model: a monotone classifier over the single feature u_{ji} cannot reorder the samples of a problem, which is why the heuristic and its trained counterpart coincide in the selection results (Section 5.1).

3.3. Execution-guided replacement

The repair component classifies syntax, name, type, assertion, import, timeout and other runtime failures, and supplies the task specification, optionally the failed program, and optionally its execution error to the replacement model. Generated code is normalized and evaluated in a fresh subprocess with a wall-clock timeout; duplicate outputs reuse execution results but still count as model calls. Candidates are ranked lexicographically by test acceptance, coarse execution progress, uniqueness, whether the program changed, and candidate order; the current program remains eligible. Because a passing candidate always ranks ahead of a failing one, a zero gap between the best passing candidate and the selected one verifies implementation consistency rather than semantic ranking ability. The principal comparisons use one round with three candidates at temperature 0.8. Fixed-budget experiments generate all candidates; allocation experiments stop at the first accepted candidate, so a failure costs one to three calls.

3.4. Allocation and accounting

Repair-All attempts every observed failure. Smart@75 ranks all initial candidates by ascending predicted pass probability and selects the lowest-scoring 75%; only selected failures are repaired. Random@75 selects the same number of initial candidates uniformly, so the two gates are *coverage-matched*, not call-matched, and repeated random replays characterize both gains and costs. Further analyses compare policy prefixes, including reversed score order and shortest-program-first order, under a common repair-call cap; because discrete trace costs can leave a remainder under a cap, attempt counts are reported beside gains. All are retrospective trace replays. The call axis measures marginal replacement-model generation only; total deployment cost would add initial sampling, uncertainty computation, probe execution, predictor fitting and test execution, for which no end-to-end latency, token or energy measurements are available, so no claim of overall computational savings is made.

4. Study design

This section fixes every component that determines what the reported numbers mean: the models and why they were chosen, the cohorts and their sizes, the control arm each experiment is read against, and the measures and statistical tests. Table 2 maps each research question to its evidence and serves as the index to Section 5.

4.1. Models, cohorts, and benchmark scope

Every model must be open-weight, because the experiments pin a seed, a numerical precision and a device and replay stored prompts months later, which a hosted checkpoint behind a stable name cannot support; this also keeps the study free of paid inference. Within that constraint each model varies one factor relative to the others.

The primary generator is CodeGen-350M-mono [40]. Its small size is the reason for the choice: it leaves a failure pool large enough to allocate a budget over (148 of 164 HumanEval and 54 of 60

MBPP candidates), which a stronger generator would not, and supports ten samples per problem on a single GPU. DeepSeek-Coder-1.3B-base [41] comes from a different family at comparable scale, so a signal that survives it is not an artifact of CodeGen's training data or tokenizer. The Qwen2.5-Coder ladder at 0.5B, 1.5B, 3B and 7B parameters [42] varies capability within one family, tokenizer and training recipe; 7B is the largest rung the available hardware accommodates, with weight offloading.

The replacement model is Qwen2.5-Coder-1.5B-Instruct [42]. Repair requires following an instruction, which base completion models are not trained to do, so the repairer is instruction-tuned out of necessity, and the instruction-tuned member of the ladder family avoids a further uncontrolled difference. The principal repair setting is therefore *cross-model*: a 350M completion model supplies failures and a larger instruction-tuned model replaces them. All three prompt arms share this replacement model, so their differences cannot be attributed to it. Larger or closed-weight repairers are out of scope. Inference ran locally or on a hosted GPU, without paid model APIs.

HE60 denotes a 60-problem HumanEval subset and HE164 all 164 problems. The MBPP cells use a fixed 60-problem cohort, the first 60 MBPP problems also present in MBPP+, so that EvalPlus probe inputs exist for every problem; their labels are the base MBPP tests unless strengthened labels are named explicitly. MBPP+ is a re-authored suite rather than a superset of the base assertions, so its counts do not decompose as base counts minus verifier false positives. The held-out repair experiment uses a separate MBPP split.

Table 2: Experiment inventory, read as the map from research question to evidence. Cohorts are analyzed separately and are not pooled into a single success rate.

Experiment	RQ	Unit and size	Principal control
Uncertainty and selection	1	60 or 164 problems, $N = 10$	Text, execution, length and runnability arms; grouped folds
BigCodeBench stress test	1	60 eligible problems, $N = 10$, five model sizes	Fixed problem set; recorded CUDA/float16 execution
Held-out repair	2	12 development and 50 evaluation failures, MBPP	Configuration selected on development failures only
Prompt ablation	2	148 HumanEval and 54 MBPP failures	Three arms at 444 and 162 calls per arm
Joint MBPP	3	60 initial candidates, 54 failures	Shared stop-on-pass repair pool; 200 random gates
Joint HumanEval	3	164 initial candidates, 148 failures	Same policy family; 200 random gates

For the held-out repair study, MBPP test tasks are shuffled with seed 42. Sixty-eight tasks yield 62 initial failures, split into 12 development and 50 evaluation failures; the evaluation pool contains 27 assertion, nine name, five runtime, five syntax, three type, and one timeout failure. Temperature and duplicate policy are selected on the development set only.

For the joint experiments, sample index zero is selected deterministically from each problem's stored ten-sample pool; the MBPP cohort initially contains six passing programs and HumanEval sixteen. Scores and repair traces are joined by source run, task, sample and label coordinates. The HumanEval replacement pool records CUDA/float16, whereas the earlier MBPP allocation pool lacks resolved device/dtype metadata, so absolute repair rates are not compared across cohorts.

The MBPP prompt ablation regenerates candidates for the same 54 coordinates in a separate run, so its code-plus-error arm (13/54) differs from the allocation pool (10/54). Supplementary Section S1 gives cohort definitions and scope qualifications.

4.2. Measures and statistical comparisons

Each measure is chosen for a property this design requires.

Correctness prediction is summarized primarily by ROC-AUC. The cohorts differ in base rate by roughly a factor of five, from about 9% positive samples on HE164 to about 23% on the easier 60-problem cells, and any summary read at a fixed threshold moves with the base rate even when the ranking is unchanged; enlarging a cohort from 60 to 164 problems lowered F_1 while raising ROC-AUC during development. ROC-AUC is threshold-free and invariant to the class prior [43], and it is the quantity the allocation experiments consume, since a gate orders candidates by score and never applies a universal cut. Because it can remain favorable while precision at a usable operating point is poor [44], precision–recall summaries are reported in the supplementary analyses, and calibration is measured separately by the expected calibration error [12].

Selection accuracy is the proportion of problems whose selected sample passes; its baseline is the expected accuracy of a uniform draw (the mean pass fraction within the pools) rather than one seeded draw. Repair success is failure-conditioned, repaired failures divided by initial failures, so that an arm is not credited for candidates it never attempted. Calls per added pass is the unit price of an improvement and is always reported with the attempt count, because two policies can reach the same gain under the same cap while attempting different numbers of failures.

Wilson intervals summarize binomial proportions because several denominators are small (50 held-out failures, 54 MBPP coordinates, cells with six or ten positives), where the normal approximation has poor coverage [45]. Arm comparisons are paired, since every arm is evaluated on identical coordinates from one frozen pool, and use the mid-p McNemar test, which behaves better than the exact conditional and asymptotic forms at the discordant counts observed here [46]; McNemar's test is the binary-outcome analogue of the paired t-test and tests the same difference without a normality assumption. Where several comparisons are reported together, p-values are adjusted with the Benjamini–Hochberg procedure, which controls the false discovery rate for multiple correlated comparisons [47]. Where a continuous quantity exists, a t-test is reported: the 200 coverage-matched random replays give a distribution of added passes, compared with the gate's observed value by a one-sample t-test (Section 5.3). Repeated gates reuse one frozen generation pool, so they measure allocation variability conditional on that pool, not variability over regenerated repairs or cohorts.

Every claim is read against a ladder of baselines, not against no intervention alone: chance, completion length and runnability for prediction; a uniform draw and the oracle for selection; coverage-matched random gating over 200 replays and an initial-length ordering for allocation; and a further generator draw at matched call count for repair. Where a free baseline wins, that is reported as the result. Numerical comparison with the external methods of Section 2 is not attempted: they use larger generators (the nearest execution-based estimators report open-weight rungs at or above the largest model used here), different benchmark subsets, test versions and sample budgets, so a difference would not be attributable to the method (Table 1).

Every reported number is derived from persisted per-sample or per-candidate records in the version-controlled repository, and later analyses (corrected representations, strengthened labels, new selectors and policies) reuse them without repeating inference. The HumanEval allocation cohort and the MBPP prompt-ablation replication were pre-registered, including their decision

rules. Parts of the analysis scripts were refactored with AI coding assistants (Claude, Anthropic; ChatGPT, OpenAI) and validated against the persisted records (Section 7.2).

5. Results

Results are reported per research question; within each, subsubsections separate findings that rest on different cohorts, representations or control arms, so that no result is read as support for a claim it does not test.

5.1. RQ1: correctness estimation and selection

5.1.1. Execution information improves sample-level ranking

Table 3 reports the corrected-representation ablation. Text–execution fusion achieves sample-level ROC-AUC between 0.88 and 0.93 across four cells; execution behavior alone is competitive, and crash rate alone is weaker.

Table 3: Sample-level ROC-AUC, corrected representation, five-fold grouped logistic models. CG = CodeGen-350M; DS = DeepSeek-Coder-1.3B. Dark blue with bold marks the best arm in each column, light blue the second best; execution information is in one of the two in every column.

Feature arm	CG/HE164	DS/HE60	CG/MBPP	DS/MBPP
Crash rate only	0.65	0.59	0.73	0.75
Text families	0.86	0.85	0.75	0.70
Execution behavior	0.90	0.92	0.83	0.88
Text + execution	**0.93**	**0.92**	**0.88**	**0.89**

At problem level the picture changes (Table 4). Two single-feature arms need no disagreement computation: the mean completion length of a problem's samples and the fraction of samples that crash on every probe. One of them meets or exceeds the text-plus-execution arm in three of four cells, completion length on CG/HE164 (0.854 against 0.851) and crash rate on CG/MBPP (0.791 against 0.683) and DS/MBPP (0.729 against 0.647); the fusion arm leads clearly only on DS/HE60, the cell with the strongest generator. Most differences are within one fold standard deviation, so at problem level the richer arms are not separated from a one-feature baseline and a problem-level deployment claim is not supported, although the sample-level claim is. The smaller number of rows (60 or 164 against 600 or 1,640) is the most economical explanation.

Table 4: Problem-level ROC-AUC with fold standard deviations; same cells, representation, folds and model as Table 3, one row per problem. Blue marks the best and second-best arm per column; amber marks a free single-feature baseline that meets or exceeds the text-plus-execution arm.

Feature arm	CG/HE164	DS/HE60	CG/MBPP	DS/MBPP
Chance	0.500	0.500	0.500	0.500
Completion length only	**.854** ±.076	.904 ±.072	.713 ±.108	.446 ±.187
Crash rate only	.786 ±.066	.723 ±.052	**.791** ±.140	**.729** ±.078
Text families	.828 ±.081	.926 ±.049	.728 ±.081	.583 ±.178
Execution behavior	.788 ±.101	.939 ±.051	.627 ±.261	.649 ±.106
Text + execution	.851 ±.044	**.969** ±.029	.683 ±.072	.647 ±.177

Useful ranking can translate into better selection (Table 5). On DS/HE60 the execution-consensus heuristic selects passing code on 42 of 60 problems (0.700), against an expected random-selection accuracy of 0.495, using existing samples only. The heuristic is at least as effective as the fused selector in three of four cells, so a higher prediction AUC need not improve selection.

Table 5: Selection accuracy from ten stored candidates per problem (separate selection arms, not a rerun of Table 3). Amber: model-free baseline; blue: best selector per column (two cells tied); grey italics: oracle ceiling, which picks a passing candidate whenever one exists. The last row is the share of the random-to-oracle gap closed by the column's best selector.

Selector	CG/HE164	DS/HE60	CG/MBPP	DS/MBPP
Random, expected	0.091	0.495	0.153	0.262
Text classifier	0.098	0.567	**0.250**	0.300
Text + execution classifier	0.122	0.650	**0.250**	**0.367**
Execution consensus	**0.140**	**0.700**	0.233	**0.367**
Oracle	*0.220*	*0.783*	*0.383*	*0.550*
Headroom captured (derived)	0.38	0.71	0.42	0.37

The last row of Table 5 is the share of the random-to-oracle gap that the best selector closes: 0.71 on DS/HE60 but only 0.37–0.42 elsewhere. Selection thus comes closest to its ceiling where the generator is strongest and the candidate pool already good.

5.1.2. Measurement controls limit the interpretation

Including the shared prompt in every compared program changes the measured lexical similarity. In CG/HE164, removing that prefix changes token-Jaccard's Pearson correlation with pass rate from −0.219 to −0.548, whereas AST-sequence correlation moves only from −0.580 to −0.581. Completion length alone correlates at −0.412; after controlling for prompt and completion lengths, the corrected token-Jaccard and AST correlations remain −0.404 and −0.434. The residuals indicate additional information, but the raw coefficients overstate what complexity buys.

Behavioral agreement must also be distinguished from runnability. On the population with co-running evidence, behavioral scores retain ROC-AUC 0.77–0.90 while the runnability-only control falls to 0.53–0.56. An implementation that drops failed probes and assigns zero distance to an unsupported pair makes all-crash samples appear confident; reporting valid-probe support and handling abnormal outcomes explicitly avoids that error.

Good ranking is not a calibrated guarantee. In the HE164 sample-level text-core arm, nested beta recalibration reduces the expected calibration error (ECE) from 0.226 to 0.016, but in an already well-calibrated 60-problem arm it increases it from 0.042 to 0.080. A stored fusion arm with ROC-AUC 0.934 reaches only 3.7% empirical coverage at a 5% selective-error threshold on CG/HE164, a retrospective operating point rather than a prospective guarantee (Supplementary Section S2).

5.1.3. Benchmark complexity changes the feature ranking

The BigCodeBench extension uses a fixed 60-problem subset drawn from the 997 of 1,140 tasks whose reference solutions pass in the recorded execution environment. CodeGen-350M and Qwen models of 0.5B, 1.5B, 3B and 7B are evaluated on it with ten samples, temperature 0.7, seed 42 and CUDA/float16; mean pass fractions rise from 0.0450 to 0.4067. AST-sequence diversity is significant after the reported correction at only two of five model sizes, token Jaccard at four, so

the most useful similarity representation depends on the task distribution. These correlations use the full-code representation, under which token-level metrics are inflated by the shared prompt (Section 5.1.2), so the ranking should be confirmed under the corrected representation; tree edit distance was not run in this extension, so the result concerns AST-sequence diversity rather than all structural metrics (Supplementary Section S3, Table S3).

Answer to RQ1: Disagreement features rank individual samples well: sample-level ROC-AUC reaches 0.88–0.93 once execution behavior is included, and execution consensus converts that ranking into better selection, 0.700 against an expected 0.495 on DS/HE60. The ranking is not robust across decision levels or task distributions: at problem level a free single-feature baseline (completion length or crash rate) meets or exceeds the full feature arm in three of four cells, and on BigCodeBench the best similarity representation changes with the benchmark. Good ranking is also not a guarantee, since a 0.934 arm supports only 3.7% coverage at a 5% error budget.

5.2. RQ2: repair gains and the value of context

5.2.1. Held-out repair and candidate diversity

The selected configuration repairs 12 of 50 held-out MBPP failures: 24.0%, with a Wilson 95% interval of 14.3–37.4%. It generates 150 candidates, of which 136 are unique; eight successes occur at the first candidate, two at the second and two at the third, so later candidates provide four of the twelve. The ranking oracle gap is zero, as expected when passing candidates receive priority. A stop-on-pass replay preserves all twelve successes while reducing calls from 150 to 132, a 12% saving in replacement calls for the observed candidate order. This held-out estimate is distinct from the joint MBPP cohort, which has a different failure pool and repair success rate.

5.2.2. Fresh generation is a stronger control than no repair

The prompt ablation holds the replacement model, temperature, seed, cohort and candidate count fixed. The *none* arm receives the problem specification, *code* adds the failing program, and *error* adds both code and traceback; each generates three candidates per failure without early stopping. Context-free generation is the strongest average policy on both cohorts (Table 6, Figure 3).

Table 6: Prompt ablation under base tests, with the matched-call regeneration baseline as the first row of each cohort. All rows of a cohort spend the same number of calls; baseline calls go to the generator (CodeGen-350M) and reveal stored samples, prompt-arm calls go to the replacement model (Qwen2.5-Coder-1.5B-Instruct), so the baseline is not matched on model. Rates condition on initial failure. p_{BH}: paired mid-p McNemar test against the cohort's baseline row, Benjamini–Hochberg adjusted within the cohort. Amber: baseline; blue: best prompt arm; the two MBPP context arms are not distinguishable from the baseline.

Cohort	Prompt arm	Repaired	Success % [95% CI]	Calls	Calls/succ.	p_{BH}
HumanEval	Resample, 3 samples	12/148	8.1 [4.7, 13.6]	444	37.0	–
	Task only (none)	**91/148**	**61.5 [53.5, 68.9]**	444	**4.88**	1.1×10^{-21}
	Task + failing code	65/148	43.9 [36.2, 52.0]	444	6.83	4.1×10^{-12}
	Task + code + error	54/148	36.5 [29.2, 44.5]	444	8.22	7.1×10^{-10}
MBPP	Resample, 3 samples	11/54	20.4 [11.8, 32.9]	162	14.73	–
	Task only (none)	**27/54**	**50.0 [37.1, 62.9]**	162	**6.00**	2.3×10^{-4}
	Task + failing code	14/54	25.9 [16.1, 38.9]	162	11.57	0.58
	Task + code + error	13/54	24.1 [14.6, 36.9]	162	12.46	0.58

The baseline rows show what repair is worth at all: revealing three further stored samples costs exactly the three calls each prompt arm spends, over the same coordinates. On HumanEval

every prompt arm beats that baseline decisively: the discordances are 81–2, 59–6 and 47–5, and all three adjusted mid-p values are below 10^{-9}. The calls go to different models, however, so the defensible reading is that a stronger instruction-tuned repairer beats three further samples of a four-times-smaller generator at matched call count, not that failure conditioning produces the gain; the baseline is also an accounting replay of stored samples rather than a new experiment.

The baseline also explains the low MBPP rates. There the free baseline already rescues 20.4% of the failures, against 8.1% on HumanEval, and the two context arms, at 25.9% and 24.1%, are *not* distinguishable from it (9–6 and 7–5 discordances, adjusted $p = 0.58$); only the context-free arm separates, at 50.0% with $p = 2.3 \times 10^{-4}$. At this generator–replacement-model pairing and call budget, conditioning on the failing MBPP program buys no more than drawing three more samples from the generator that wrote it. With 54 coordinates and a Wilson interval of [16.1, 38.9] on the strongest context arm, this excludes a large advantage for context on MBPP, not a small one.

Table 7 locates the cohort difference in the failure mix rather than in a cohort-wide repair difficulty. Assertion errors, where the program runs and returns a wrong answer, are 116 of 148 HumanEval failures (78.4%) but 33 of 54 MBPP failures (61.1%), and almost the whole context-free advantage sits there: 73 against 46 on HumanEval, 27 of the arm's 37-case lead, and 18 against 10 on MBPP, 8 of a 14-case lead. Syntax errors are never repaired by either context arm in either cohort (0 of 5 and 0 of 3), against the expectation that a traceback helps most where a program does not parse; at these denominators that is a hypothesis for a larger cohort rather than a result. The traceback arm leads only on HumanEval runtime errors, 5 of 8 against 4 of 8, which is one coordinate.

Table 7: Successful repairs by failure category of the initial program, per prompt arm, under base tests. The failure count is a property of the initial pool and is identical across arms. Bold marks the best arm within a row; the context-free arm leads or ties in nine of the ten rows.

Cohort	Failure category	Failures	None	Code	Code+error
HumanEval	Assertion error	116	**73**	57	46
	Name error	15	**9**	3	2
	Runtime error	8	4	4	**5**
	Syntax error	5	**3**	0	0
	Type error	4	**2**	1	1
MBPP	Assertion error	33	**18**	8	10
	Name error	7	**4**	**4**	2
	Runtime error	4	**2**	1	0
	Syntax error	3	**1**	0	0
	Type error	7	**2**	1	1

On HumanEval, the paired discordances favoring task-only over code-plus-error are 49 versus 12, with adjusted mid-$p = 3.6 \times 10^{-6}$. `HumanEval/32` has no usable EvalPlus entry and is excluded from both the base and the strengthened labels, so every strengthened-label rate is computed over 147 coordinates rather than 148; under the strengthened labels the counts are 47 versus 14 and the adjusted value is 5.2×10^{-5}. The code-only versus code-plus-error contrast is also significant on HumanEval; on MBPP task-only improves on both context arms, but code-only and code-plus-error are not statistically distinguished. The additional harm attributable specifically to traceback is therefore not a replicated two-cohort finding.

Context is not useless for every problem: the union of the task-only and code-only arms repairs

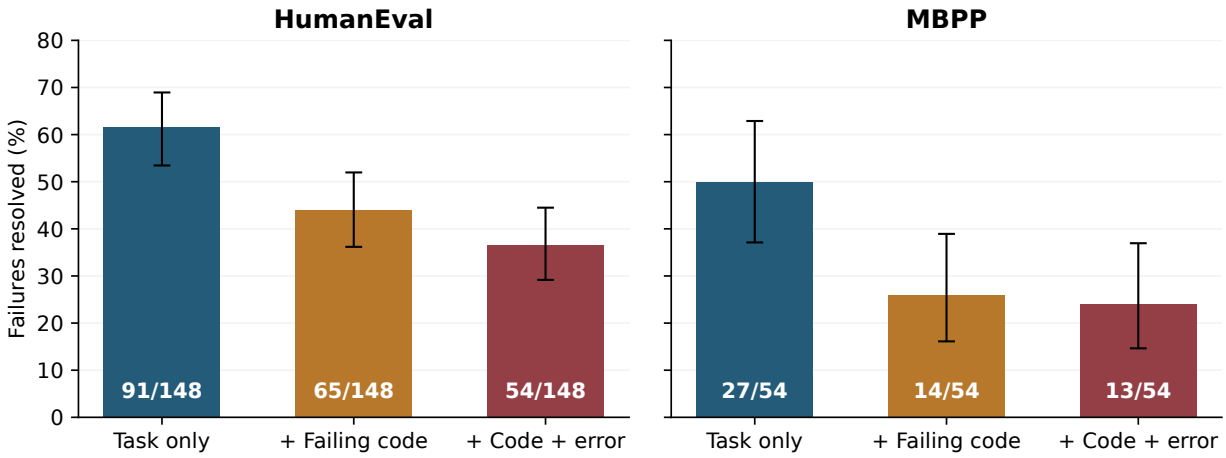


Figure 3: Failure-conditioned success under matched call budgets. Error bars are Wilson 95% intervals computed from the reported counts; they are descriptive and do not replace paired tests.

109 HumanEval cases, eighteen more than task-only alone, and the full-context arm adds none outside it. An evaluated per-problem router does not realize this headroom: its out-of-sample coverage choice produces 85 successes instead of the default's 91, and because it reuses upstream predictions it is retained as an exploratory negative result, not proof that useful routing is impossible. The pattern is consistent with a failed program constraining the replacement model to unproductive edits, but is not a causal demonstration of anchoring or of a capability-gap effect (Supplementary Section S4).

Answer to RQ2: Repair is worth its calls, but failure context is not. Against a matched-call baseline that reveals three further stored generator samples, every HumanEval prompt arm wins decisively (adjusted $p < 10^{-9}$), while on MBPP only the context-free arm separates from it and the two context arms do not ($p = 0.58$); because those calls go to a stronger instruction-tuned model, the win is matched on calls and coordinates, not on model. Among the arms, giving the replacement model the failing program costs success at a fixed call budget: 61.5% against 43.9% and 36.5% on HumanEval, 50.0% against 25.9% and 24.1% on MBPP. The loss is concentrated in assertion errors, and 18 HumanEval coordinates are still solved only with the failing program, so context is uniformly misapplied at this generator–repairer pairing, not useless.

5.3. RQ3: allocating repair effort

5.3.1. A useful failure score can yield an inefficient gate

In the joint MBPP experiment (Table 8), Repair-All raises final accuracy from 6/60 to 16/60, and Smart@75 retains seven of the ten additional passes while saving 27 repair calls, which alone could suggest a useful integration. The random control changes the interpretation: its nine gains cost 106 calls, against seven gains for 120 calls under Smart@75.

The gate selects more actual failures, 44 of its 45 candidates against 40 under the seeded random gate, but initial passes cost no repair calls, so this precision increases expenditure, and it is insufficient if the extra failures are not more repairable. Paired final-outcome differences are not resolved in this 60-problem cohort, so the conclusion is a lack of demonstrated allocation advantage, not superiority of one random seed.

Table 8: Joint MBPP allocation. Smart and random gates each select 45 initial candidates; the numbers of selected failures and actual repair calls differ.

Policy	Final passes	Added	Attempts	Calls	Calls/gain
No repair	6/60	0	0	0	–
Repair-All	16/60	10	54	147	14.70
Smart@75	13/60	7	44	120	17.14
Random@75, seed 42	15/60	9	40	106	11.78

Two hundred random replays test whether that seed is unusual. On MBPP, random gates add 7.56 ± 1.19 passes for 110.1 ± 3.5 calls (mean $\pm$ standard deviation), against Smart@75's seven passes and 120 calls, and in 82.5% of draws achieve at least as many gains at no greater call cost. On HumanEval they add 40.5 ± 2.7 gains for 276.1 ± 5.9 calls, against 36 and 312, and dominate or tie in 97.5% of draws. A one-sample t-test of the replay distribution against the gate's observed value agrees, $t(199) = 6.66$, $p = 2.7 \times 10^{-10}$ on MBPP and $t(199) = 23.57$, $p = 1.7 \times 10^{-59}$ on HumanEval, in both cases because the random mean exceeds the gate. That test characterizes the replay distribution over one frozen pool rather than a population of cohorts, so the dominance percentages remain the primary summary.

5.3.2. The relationship with repairability is cohort-dependent

The same stored pass score is evaluated against initial correctness and against repair success among initial failures (Table 9). On MBPP the repairability AUC is near chance, but its wide interval establishes neither independence nor the absence of a useful signal. On HumanEval, higher predicted pass probability goes with greater repair success even among failures, so ascending pass-score order allocates effort in the unfavorable direction on that cohort.

Table 9: One score, two prediction targets. Initial-correctness rows use sample index zero, not all ten samples. Repair-success rows condition on initial failure.

Cohort	Target	n / positives	ROC-AUC	Bootstrap 95%
MBPP	Initial pass	60 / 6	0.944	[0.804, 1.000]
HumanEval	Initial pass	164 / 16	0.947	[0.882, 0.993]
MBPP	Repair success	54 / 10	0.518	[0.298, 0.743]
HumanEval	Repair success	148 / 54	0.706	[0.616, 0.793]

Because the three prompt arms of Section 5.2 cover the same 148 HumanEval coordinates, the allocation question can be answered as a manipulation rather than as a single measurement (Table 10). The two orderings run opposite ways: cost-effectiveness falls monotonically as failure context is added, from 4.88 to 8.22 calls per added pass, while the score's ranking of repair success rises monotonically over the same arms, from 0.542 at chance to 0.706. The arm in which the gate carries a usable allocation signal is therefore the arm that should not be deployed, and the arm that should be deployed is the one in which the gate is uninformative. This supplies the mechanism the composition argument above could only infer: a replacement conditioned on the failing program inherits its near-miss structure, which a correctness score measures, whereas one not conditioned on it inherits only the problem's difficulty. On MBPP the context-free arm reproduces the chance result, at 0.481 with a bootstrap interval of [0.322, 0.639] on 27 positives, and the code-only arm reaches 0.675 ($p = 0.054$, 14 positives); the MBPP traceback arm is omitted because its only

allocation pool is not the pool behind Table 6 and carries ten positives. The cost ordering replicates on MBPP, at 6.00, 11.57 and 12.46 calls per added pass.

Table 10: HumanEval, 148 coordinates: what one stored pass score is worth inside each prompt arm. Calls per success is from Table 6 (fixed budget, three candidates per failure, 444 calls per arm); ROC-AUC is against repair success among initial failures, with a label-permutation p over 10,000 permutations and a bootstrap interval over 10,000 resamples. Amber marks the cheapest arm, blue the arm the score ranks best: opposite ends of the same ladder. The bottom row repeats Table 9's HumanEval repair-success row.

Prompt arm	Repaired	Calls/success	ROC-AUC	Permutation p	Bootstrap 95%
Task only (none)	91/148	4.88	0.542	0.39	[0.444, 0.636]
Task + failing code	65/148	6.83	0.648	0.0016	[0.556, 0.737]
Task + code + error	54/148	8.22	**0.706**	1.0×10^{-4}	[0.616, 0.793]

5.3.3. Simple allocation rules are competitive under call caps

The call budgets here are stop-on-pass budgets, the convention an allocation policy would operate under, so the code-plus-error arm costs 368 calls rather than the 444 of Table 6, over the same coordinates and stored candidates; candidate seeds are index-deterministic, so both counts are exact. Reversing the score order on HumanEval recovers 36 additional passes in 146 calls, compared with the 312 calls used by Smart@75 for the same gain. The direction was chosen after inspecting this cohort, so it is an explanatory counterfactual rather than a validated deployment improvement. Sorting by initial character count is also competitive. Figure 4 plots gains at four fractions of the 368-call Repair-All budget.

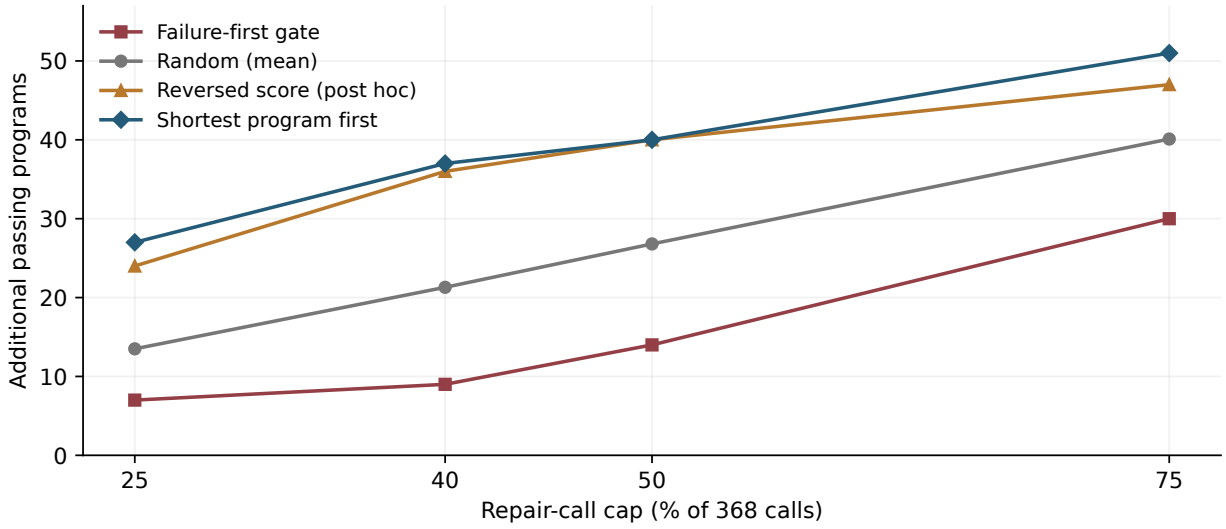


Figure 4: HumanEval allocation replays under common repair-call caps. Lines connect four reported operating points, not continuous measurements. The reversed-score and length analyses are exploratory. Random values are means over 200 replays.

At a 50% call cap, shortest-first and reversed-score order each obtain 40 added passes in 80 attempted failures; ascending pass-score order obtains 14 in 67 attempts, and random averages 26.8 in approximately 74. At the 25% cap, shortest-first obtains 27 gains in 45 attempts and reversed

order 24 in 42, a difference that cannot be credited solely to ranking because the attempt counts differ. These results are conditional on the HumanEval code-plus-error pool: the MBPP results do not support a universal shortest-first policy, and all these orderings allocate an arm that is weaker on average than fresh generation. Optimizing the order of an inferior intervention does not establish the best overall system (Supplementary Section S5).

Answer to RQ3: A score that ranks initial correctness at ROC-AUC 0.944–0.947 does not thereby allocate repair well. Coverage-matched random gates match or beat the uncertainty gate at no greater call cost in 82.5% of MBPP and 97.5% of HumanEval replays, and add more passes on average ($t(199) = 6.66$ and 23.57). The training target and the use target differ: ranking repairability is at chance on MBPP (0.518) and runs in the direction opposite to the gate's on HumanEval (0.706), and that signal exists only in the context arms, which are the expensive ones. Shortest-program-first, which needs no model, is competitive at every call cap on HumanEval. The conclusion is a lack of demonstrated allocation advantage in these cohorts, not that uncertainty gating cannot work.

6. Discussion

The discussion is organized around the three uses of one uncertainty score that the results separate (Figure 1). Section 6.1 explains why they must be validated separately, Sections 6.2–6.4 state what the evidence supports for each, and Section 6.5 draws implications for practice; each subsection points to the result it interprets rather than repeating it.

6.1. Three uses of uncertainty, three validation targets

The same disagreement-based score served three decisions, the (i)–(iii) of the abstract's Objectives, each with its own population, outcome and free baseline:

(i) **First use: correctness prediction**, over every generated candidate, with the initial test verdict y as outcome and chance, completion length and crash rate as baselines (RQ1).

(ii) **Second use: candidate selection**, over problems, with the pass status of the returned program as outcome, bounded by a uniform draw from the same set and by the oracle choice (RQ1).

(iii) **Third use: repair allocation**, over failing candidates only, with $z \mid y = 0$ as outcome and a random allocation of the same coverage, an ordering by a free property of the failing program, and fresh generation at the same call count as baselines (RQ2 and RQ3).

Success on the first use is relevant to the other two but no substitute for evaluating them, and the results show the three coming apart within one cohort: the score that ranks initial correctness at ROC-AUC 0.944 and 0.947 ranks repair success at 0.518 and 0.706, and the execution-consensus heuristic selects at least as well as the fused classifier in three of four cells although the fused arm is the strongest or joint-strongest predictor in all four. An uncertainty method should therefore be reported against the target of the decision it drives, and a strong result at one use should not be cited as support for another. Each control challenges a specific use: length, runnability and BigCodeBench the first; the uniform draw and the oracle bound the second; fresh generation and random allocation the third. They support an empirical contribution without a claim of benchmark leadership (Table 1 is positioning, not a leaderboard).

6.2. First use: correctness prediction

The evidence supports sample-level ranking: once execution behavior is included the fused arm is best or joint-best in every cell and execution alone is best or second best (Table 3), consistent with concurrent code-specific estimators that go beyond surface text through execution behavior [10, 11, 25] or a judged functional equivalence [24]. It does not support a problem-level deployment claim, or a claim that the richer feature arms produce the ranking: at problem level a free single feature is not separated from them (Table 4), the shared prompt inflates the lexical family's correlations, and behavioral agreement must be separated from runnability, which alone ranks barely above chance (Section 5.1.2). Ranking is also not a guarantee: the same recalibration procedure moved calibration error in opposite directions in two arms, and a high ROC-AUC supported little coverage at a strict error budget, which is the selective-prediction reading of the same scores [28]. The first use produces an ordering whose value has to be established by the decision that consumes it.

6.3. Second use: candidate selection

Selection is where the ordering converts most directly into a benefit, with no trained model, no expected outputs and no generation beyond the existing pool (Table 5). This places the result in the execution-reranking line of work [29, 30, 26] rather than the uncertainty line, and it is the use for which the evidence is most favorable. Two qualifications follow. Prediction quality and selection quality are different properties: a monotone model over a single feature cannot reorder the samples of a problem (Section 3.2), so the extra features that raise ROC-AUC need not change which candidate is returned. And the benefit depends on the pool, being largest where the generator was already strong. Selection is also inexpensive only at the margin: reusing ten existing candidates and signatures to select, or to score failures for allocation, costs little, but generating nine more for that purpose is a different economic decision that the present data do not price.

6.4. Third use: repair allocation

Allocation is the use for which the evidence is least favorable, because it has a different target. A useful conceptual objective is expected gain per unit cost,

$$V_j = \frac{\Pr(z_{j0} = 1 \mid y_{j0} = 0, \text{ available features})}{\mathbb{E}[c_j \mid \text{available features}]}, \tag{3}$$

rather than initial failure probability alone; the expression describes the decision problem and is not an estimator evaluated in this study. The HumanEval reversal shows that a less confident failure is not necessarily a more valuable repair target, and the MBPP interval shows that a correctness score's relation to repairability can remain unresolved. The arm-level manipulation (Table 10) locates the allocation signal in the context-bearing arms, the expensive ones, so improving the correctness predictor would not by itself repair the allocation result: the third use needs a target defined on the repair outcome.

The repair step being allocated must also earn its place against regeneration. Comparing repair only with no repair conflates additional sampling, a different model and feedback conditioning; the fixed-call ablation removes part of that ambiguity by holding the replacement model fixed, and task-only regeneration wins on both cohorts (Section 5.2). This does not invalidate execution-guided repair as a field [9, 35]; it shows that, for the studied failures and this cross-model pairing, adding the failed code is not a reliable default improvement over regeneration. Eighteen HumanEval failures still need a context-bearing arm, but the evaluated router shows the gap between available headroom and a practical decision rule.

6.5. Implications for industrial practice

LLM code assistants are in daily industrial use, and the rate at which developers accept their suggestions has been found to drive how productive developers perceive them to be [2]. Developers report that the most important reason for not using them is output that does not meet functional or non-functional requirements [48], and generated code can carry security weaknesses that a functional test does not reveal [49]. Industrial pipelines therefore already filter LLM output by execution: repair at scale validates fixes by tests before proposing them [32], and LLM-based test improvement keeps a test only if it builds, passes reliably and adds coverage [50]. Uncertainty-guided selection and repair allocation extend that pattern; the three uses yield three recommendations on evaluating such a policy before adoption, not deployment recommendations, since the evidence is retrospective and limited to small open-weight models on bounded Python tasks.

For the first use, treat a correctness score as a triage ordering, not as an acceptance gate. A team that wants to auto-accept or route generated code for review should measure coverage at its own tolerated error rate on its own code, recalibrate on its own distribution, and check first whether a free feature such as completion length or crash rate already achieves the same ranking.

For the second use, execution consensus is the cheapest benefit if several candidates are already produced. Services that already sample and sandbox several candidates, for example for best-of-n generation, can select by behavioral agreement without a trained model or expected outputs, and should judge the gain by the share of the random-to-oracle gap it closes on their own tasks rather than by raw accuracy.

For the third use, replay before deploying. Before an uncertainty-gated repair loop or retry policy is adopted, the per-attempt outcomes and call counts of one ungated run, including a regeneration arm without the failing program, should be logged and frozen. The uncertainty gate, a random gate of the same coverage and a shortest-program-first ordering can then be replayed over identical records at no further inference cost, as was done here, and the comparison should charge the cost of the uncertainty score itself: initial sampling, probe execution and predictor fitting.

7. Threats to validity

Threats are grouped by the four-way classification customary in empirical software engineering [51], with execution safety added because it constrains how the study should be reproduced. Each threat is paired with its mitigation or, where none was possible within this study, with what would be required.

7.1. Construct validity

Passing the available tests is not semantic correctness, and candidates are accepted by the same suites, which risks test overfitting. Mitigation: the prompt-ablation programs were relabeled against the strengthened EvalPlus suites and both label sets are reported (Section 5.2, Supplementary Section S4), with the one problem lacking an EvalPlus entry excluded from both; the arm ordering and the HumanEval significance survive. Other analyses state that they use the base suites. Relabeling is a stronger check, not a proof of correctness.

Probe features could leak the label. Mitigation: probes are argument lists only, so no behavioral feature reads expected values or verdicts. Benchmark-derived inputs still expose part of the evaluation input distribution; removing this would require independently fuzzed probes.

Model calls are not total cost. The prompt ablation fixes the call count, not token volume or execution time, and prompts differ in length. Mitigation: attempt counts accompany every call

count, the stop-on-pass and fixed-budget conventions are stated wherever they are used, and no claim of overall computational savings is made; token, latency and energy measurements would require an instrumented live service.

The same metric can be measured over different inputs, because tables use different representations or label conventions. Mitigation: the shared-prompt confound was measured directly (Section 5.1.2); every table names its representation, cohort and label; and ROC-AUC is complemented by calibration error and coverage at a fixed error budget, because ranking quality is not a usable operating point. Similarity to reference solutions cannot establish the absence of pretraining contamination; this is reported but not mitigated.

7.2. Internal validity

Policy differences could come from generation randomness rather than the allocation decision. Mitigation: every policy is replayed over one frozen pool with one stored outcome and call count per failing coordinate, so comparisons are paired, and the analyses assert that coordinate selections match their recorded checksums and that compared arms share one coordinate list. The traces inherit the correctness of the harness; the HumanEval allocation analyses were re-run on a second platform from the stored artifacts and reproduced every figure exactly, while the remaining analyses were run on one platform.

Predictive accuracy could be inflated by leakage. Mitigation: problem-grouped folds and nested calibration (Section 3.2). The exploratory router reuses upstream out-of-fold scores whose training partitions are not fully nested within its folds, and is reported as exploratory for that reason.

The regeneration baseline changes the model as well as the input. Mitigation: it is matched on coordinates and call count, and the caption of Table 6 and the answer to RQ2 state that it is not matched on model. Between the prompt arms the replacement model, temperature, seed, cohort and candidate count are fixed, so their differences are attributable to the prompt, though not uniquely to anchoring, model capability or traceback quality, which would require controlled repairer scales and prompt interventions.

Post-hoc choices could manufacture an effect. Mitigation: the HumanEval allocation cohort and the MBPP prompt-ablation replication were pre-registered, including their decision rules; the held-out repair configuration was selected on development failures only; and the reversed-score and length orderings, whose direction was chosen after inspecting the cohort, are labeled exploratory counterfactuals.

Device and numerical precision could differ between runs. Mitigation: the resolved device and precision are recorded wherever the run supports it; the earlier MBPP allocation pool lacks that metadata, so cross-cohort differences in absolute repair rates are not interpreted.

7.3. External validity

The experiments cover selected Python tasks, small open-weight models, one principal replacement model and mostly one generation seed per cell. Mitigation: findings are checked across two benchmarks, two generator families at comparable scale and a single-family ladder from 0.5B to 7B parameters; the allocation experiment was repeated on a second, pre-registered cohort, on which the dominance of random allocation replicated while the score–repairability relation did not (Section 5.3); and BigCodeBench stress-tests the uncertainty findings. These checks bound the conclusions rather than generalize them: benchmark failures are not representative of production defects, BigCodeBench eligibility conditions its subset on the execution environment, and model variants at different precision are not independent families.

7.4. Conclusion validity

Several denominators are small: the 50-case repair estimate has a wide interval, and the MBPP repairability analysis has ten positive outcomes. Mitigation: Wilson intervals, bootstrap intervals with label-permutation p-values, fold standard deviations, mid-p McNemar tests and Benjamini–Hochberg adjustment (Section 4.2); where a test does not reject, the text states which effect size is excluded rather than claiming equivalence.

Randomized policies could be judged on one lucky draw. Mitigation: following the recommendation to assess randomized algorithms in software engineering over many runs [52], the random gate is evaluated over 200 replays with a one-sample test against the gate. The replays reuse one frozen generation pool, so their domination percentages are conditional descriptive results, and the exploratory policies still require independent development and evaluation cohorts. Likewise, an operating point read from an observed risk–coverage curve is an empirical point, not a guaranteed future risk bound.

7.5. Execution safety

Generated programs can attempt filesystem or network operations. Mitigation: every program runs in a separate subprocess with a wall-clock timeout in a temporary working directory. These measures are not a hardened sandbox; reproduction should use isolated, disposable environments with appropriate resource and access restrictions, and the current execution design is not suitable for unrestricted deployment on a trusted host.

8. Conclusion

We integrated uncertainty estimation and execution-guided repair to study when correctness information improves decisions about additional computation. Behavioral disagreement supports sample-level prediction, at ROC-AUC 0.88–0.93 once execution is included, and candidate selection, where execution consensus returns a passing program on 0.700 of DS/HE60 problems against an expected 0.495; representation, length and runnability controls materially qualify both results. Multi-candidate repair corrects 12 of 50 held-out MBPP failures, yet task-only regeneration outperforms the tested feedback prompts under matched call budgets on both cohorts, 61.5% against 36.5% on HumanEval and 50.0% against 24.1% on MBPP.

The central integration finding is specific. The stored score that ranks initial correctness at ROC-AUC 0.944 on MBPP and 0.947 on HumanEval ranks repair success among the failures at only 0.518 and 0.706, and the gate built on it is dominated by random allocation of the same coverage. Smart@75 added 7 passes for 120 repair calls on MBPP against a random mean of 7.56 passes for 110.1 calls, and 36 for 312 on HumanEval against 40.5 for 276.1; over 200 replays a random gate matched or exceeded its gain at no greater call cost in 82.5% of MBPP and 97.5% of HumanEval draws. On MBPP the score does not resolve repairability, whereas on HumanEval it carries a signal opposite to the direction in which the gate consumes it, present only in the context-bearing arms, which cost 6.83 and 8.22 calls per success against 4.88 for task-only regeneration. A model-free shortest-program-first ordering is competitive at every call cap on HumanEval, but the MBPP results do not support it as a general rule. Reliable code generation therefore requires evaluating the full decision, including replacement strategy, allocation target, test oracle and resource budget, and these retrospective findings should be validated prospectively before they inform a deployment policy.

Several directions follow for this team and for anyone continuing the work. The most direct is to estimate the allocation objective V_j itself, training a repairability predictor on repair outcomes rather than initial correctness, with every upstream score refitted inside the evaluation folds, and to test it together with shortest-program-first on a third cohort that played no part in choosing either. A second is a repairer capability ladder at matched device and precision, testing whether regeneration's advantage over context-bearing repair narrows as the replacement model grows relative to the generator. A third is to replace benchmark-derived probes with independently fuzzed inputs, as in DiffCodeGen [26]. A fourth is a matched reimplementation of the nearest execution-based uncertainty estimators [10, 11], with execution-free functional-equivalence scoring [24] as a contrasting baseline, under one model, oracle, sample budget and environment. Beyond this setting, the three-use protocol applies to repository-level tasks such as SWE-bench [18], other languages and agentic retry loops, and a live service with token, latency and energy accounting is needed to price any policy end to end. For other researchers, the frozen repair pools are a reusable testbed: a new allocation policy can be evaluated against the gate, random allocation and the free orderings without running any model.

CRediT authorship contribution statement

Melih Elibol: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft. Primary responsibility for uncertainty estimation, calibration, candidate selection, and the uncertainty measurement analyses. **Ayşenur Gönül:** Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft. Primary responsibility for execution-guided repair, failure-pool construction, candidate ranking, and repair evaluation. Shared infrastructure and the uncertainty–repair integration are joint contributions.

Sushant Kumar Pandey: Conceptualization, Methodology, Supervision, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All data and code needed to reproduce every number in this paper are contained in the project repository, including the per-sample and per-candidate result records in `results/`, which is version-controlled rather than regenerated. Supporting tables are supplied as supplementary material, and Supplementary Section S6 gives reproduction details. An archival copy will be deposited and its DOI supplied at acceptance. No proprietary data, paid API or closed-weight model is used anywhere in the study.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) and ChatGPT (OpenAI) to help draft, shorten and edit prose, to refactor analysis scripts, and to cross-check

reported numbers against the persisted result files. The authors reviewed and edited all content and take full responsibility for the content of the published article. No generative model contributed experimental results; every reported number is recomputable from the version-controlled artifacts.

Acknowledgements

This work was carried out during a summer research internship at the University of Groningen.

References

[1] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large Language Models for Software Engineering: A Systematic Literature Review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, 2024.

[2] A. Ziegler, E. Kalliamvakou, X. A. Li, A. Rice, D. Rifkin, S. Simister, G. Sittampalam, and E. Aftandilian, "Measuring GitHub Copilot's Impact on Productivity," *Communications of the ACM*, vol. 67, no. 3, pp. 54–63, 2024.

[3] M. Chen et al., "Evaluating Large Language Models Trained on Code," *arXiv:2107.03374*, 2021.

[4] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation," in *Advances in Neural Information Processing Systems*, 2023.

[5] J. Austin et al., "Program Synthesis with Large Language Models," *arXiv:2108.07732*, 2021.

[6] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," in *Proceedings of EMNLP*, 2023.

[7] Z. Lin, S. Trivedi, and J. Sun, "Generating with Confidence: Uncertainty Quantification for Black-Box Large Language Models," *Transactions on Machine Learning Research*, 2024.

[8] A. Madaan et al., "Self-Refine: Iterative Refinement with Self-Feedback," in *Advances in Neural Information Processing Systems*, 2023. *arXiv:2303.17651.*

[9] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching Large Language Models to Self-Debug," in *Proceedings of ICLR*, 2024. *arXiv:2304.05128.*

[10] W. He, A. Sharma, and C. David, "Using Semantic Distance to Estimate Uncertainty in LLM-Based Code Generation," *arXiv:2605.09023*, 2026.

[11] X. Ren, Y. Xue, L. Ma, and Y. Huang, "Code-MUE: Measuring Code LLMs' Uncertainty through Execution-based Semantic Interaction Graphs," in *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2026. *arXiv:2607.12273.*

[12] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *Proceedings of ICML*, 2017.

[13] M. Kull, T. Silva Filho, and P. Flach, "Beta Calibration: A Well-Founded and Easily Implemented Improvement on Logistic Calibration for Binary Classifiers," in *Proceedings of AISTATS*, 2017.

[14] M. Monperrus, "Automatic Software Repair: A Bibliography," *ACM Computing Surveys*, vol. 51, no. 1, Article 17, 2018.

[15] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is Self-Repair a Silver Bullet for Code Generation?," in *Proceedings of ICLR*, 2024. *arXiv:2306.09896*.

[16] B. Stroebl, S. Kapoor, and A. Narayanan, "The Limits of Inference Scaling Through Resampling," *arXiv:2411.17501*, 2024.

[17] T. Y. Zhuo et al., "BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions," in *Proceedings of ICLR*, 2025. *arXiv:2406.15877*.

[18] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," in *Proceedings of ICLR*, 2024.

[19] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in *Proceedings of ICLR*, 2023.

[20] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation," in *Proceedings of ICLR*, 2023.

[21] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting Hallucinations in Large Language Models Using Semantic Entropy," *Nature*, vol. 630, no. 8017, pp. 625–630, 2024.

[22] Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in *Findings of EMNLP*, 2020.

[23] S. Zhou, U. Alon, S. Agarwal, and G. Neubig, "CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code," in *Proceedings of EMNLP*, 2023.

[24] D. Bouchard, M. S. Chauhan, Z. Ahmad, and H.-K. Ra, "Functional Entropy: Predicting Functional Correctness in LLM-Generated Code with Uncertainty Quantification," in *Proceedings of EMNLP*, 2026, to appear. *arXiv:2605.28500*.

[25] Y. Shi, C. Zhang, Y. Li, H. Wang, Y. Chen, N. Collier, and X. Gu, "Code Is More Than Text: Uncertainty Estimation for Code Generation," *arXiv:2606.09577*, 2026.

[26] Y. He, E. Wang, J. Wang, X. Ouyang, and H. Chen, "Code Generation by Differential Test Time Scaling," *arXiv:2605.20473*, 2026.

[27] C. Spiess, D. Gros, K. S. Pai, M. Pradel, M. R. I. Rabin, A. Alipour, S. Jha, P. Devanbu, and T. Ahmed, "Calibration and Correctness of Language Models for Code," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025.

[28] J. Traub, T. J. Bungert, C. T. Lüth, M. Baumgartner, K. H. Maier-Hein, L. Maier-Hein, and P. F. Jäger, "Overcoming Common Flaws in the Evaluation of Selective Classification Systems," in *Advances in Neural Information Processing Systems*, 2024.

[29] F. Shi, D. Fried, M. Ghazvininejad, L. Zettlemoyer, and S. I. Wang, "Natural Language to Code Translation with Execution," in *Proceedings of EMNLP*, pp. 3533–3546, 2022.

[30] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "CodeT: Code Generation with Generated Tests," in *Proceedings of ICLR*, 2023.

[31] C. Le Goues, M. Pradel, and A. Roychoudhury, "Automated Program Repair," *Communications of the ACM*, vol. 62, no. 12, pp. 56–65, 2019.

[32] A. Marginean, J. Bader, S. Chandra, M. Harman, Y. Jia, K. Mao, A. Mols, and A. Scott, "SapFix: Automated End-to-End Repair at Scale," in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 269–278, 2019.

[33] C. S. Xia, Y. Wei, and L. Zhang, "Automated Program Repair in the Era of Large Pre-trained Language Models," in *Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023.

[34] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated Repair of Programs from Large Language Models," in *Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023.

[35] C. S. Xia and L. Zhang, "Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pp. 819–831, 2024.

[36] K. Zhang and D. Shasha, "Simple Fast Algorithms for the Editing Distance between Trees and Related Problems," *SIAM Journal on Computing*, vol. 18, no. 6, pp. 1245–1262, 1989.

[37] D. R. Roberts, V. Bahn, S. Ciuti, M. S. Boyce, J. Elith, G. Guillera-Arroita, S. Hauenstein, J. J. Lahoz-Monfort, B. Schröder, W. Thuiller, D. I. Warton, B. A. Wintle, F. Hartig, and C. F. Dormann, "Cross-Validation Strategies for Data with Temporal, Spatial, Hierarchical, or Phylogenetic Structure," *Ecography*, vol. 40, no. 8, pp. 913–929, 2017.

[38] S. Kapoor and A. Narayanan, "Leakage and the Reproducibility Crisis in Machine-Learning-Based Science," *Patterns*, vol. 4, no. 9, Article 100804, 2023.

[39] G. C. Cawley and N. L. C. Talbot, "On Over-Fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation," *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010.

[40] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis," in *International Conference on Learning Representations*, 2023. *arXiv:2203.13474*.

[41] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, et al., "DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence," *arXiv:2401.14196*, 2024.

[42] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, et al., "Qwen2.5-Coder Technical Report," *arXiv:2409.12186*, 2024.

[43] A. P. Bradley, "The Use of the Area under the ROC Curve in the Evaluation of Machine Learning Algorithms," *Pattern Recognition*, vol. 30, no. 7, pp. 1145–1159, 1997.

[44] J. Davis and M. Goadrich, "The Relationship between Precision-Recall and ROC Curves," in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 233–240, 2006.

[45] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[46] M. W. Fagerland, S. Lydersen, and P. Laake, "The McNemar Test for Binary Matched-Pairs Data: Mid-p and Asymptotic Are Better than Exact Conditional," *BMC Medical Research Methodology*, vol. 13, Article 91, 2013.

[47] Y. Benjamini and Y. Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing," *Journal of the Royal Statistical Society B*, vol. 57, no. 1, pp. 289–300, 1995.

[48] J. T. Liang, C. Yang, and B. A. Myers, "A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024.

[49] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in *Proceedings of the IEEE Symposium on Security and Privacy (SP)*, 2022.

[50] N. Alshahwan, J. Chheda, A. Finogenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, "Automated Unit Test Improvement using Large Language Models at Meta," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE)*, pp. 185–196, 2024.

[51] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*, Springer, Berlin, Heidelberg, 2012.

[52] A. Arcuri and L. Briand, "A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering," in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, pp. 1–10, 2011.